

Part A: Fundamentals — Answers

Data Analysis, Data Science & Machine Learning — Question Set

1. Short Note: What is Data Analysis?

Data Analysis is the process of inspecting, cleaning, transforming, and modeling data with the goal of discovering useful information, drawing conclusions, and supporting decision-making. It involves collecting raw data from various sources, organizing it, applying statistical and computational techniques, and interpreting the results to identify patterns, trends, and relationships.

In practice, data analysis typically includes descriptive analysis (summarizing what has happened), diagnostic analysis (understanding why it happened), predictive analysis (forecasting what might happen), and prescriptive analysis (recommending actions). It is a core step in every data-driven field, including business intelligence, data science, and machine learning.

2. Planning a Data Science Project — Steps

A data science project is typically planned and executed through the following structured stages:

- 1 **Problem Definition:** Clearly understand the business question and define measurable goals and success metrics.
- 2 **Data Collection:** Gather relevant data from databases, APIs, files, or web sources.
- 3 **Data Cleaning:** Handle missing values, duplicates, outliers, and inconsistent formats.
- 4 **Exploratory Data Analysis (EDA):** Use statistics and visualizations to understand distributions and relationships.
- 5 **Feature Engineering:** Create, transform, or select variables that improve model performance.
- 6 **Model Building:** Select and train suitable machine learning algorithms.
- 7 **Model Evaluation:** Assess performance using metrics such as accuracy, precision, recall, or RMSE.
- 8 **Deployment:** Integrate the model into a production system or application.
- 9 **Monitoring & Maintenance:** Track model performance over time and retrain as data evolves.



Figure: End-to-end Data Science project pipeline.

3. Machine Learning Problem Statement — Customer Churn

Problem: Predict whether a customer will churn (stop using the service) based on their purchase behavior.

Type: This is a **Supervised Binary Classification** problem, since the target variable 'Churn' has two possible outcomes: Yes (1) or No (0).

Input (X): Purchase-behavior features such as purchase frequency, average order value, recency of last purchase, total spend, number of support tickets, subscription tenure, and discount usage.

Output (y): A binary label — 1 if the customer churns, 0 if the customer stays.

Objective: Learn a function $f(X)$ that estimates the probability of churn, commonly modeled with an algorithm such as Logistic Regression, Random Forest, or XGBoost:

$$\hat{y} = f(X) \in \{0, 1\}$$
$$P(\text{Churn} = 1 \mid X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 X_1 + \dots + \beta_n X_n)}}$$

Evaluation metrics: Accuracy, Precision, Recall, F1-score, and ROC-AUC — with Recall often prioritized since failing to catch a churning customer is costlier than a false alarm.

4. Tensors — Explained in Depth

What are Tensors?

A **Tensor** is a generalized, multi-dimensional array of numbers, and it is the fundamental data structure used in machine learning and deep learning frameworks (such as NumPy, TensorFlow, and PyTorch). A tensor is defined by its **rank** (number of axes/dimensions), **shape** (size along each axis), and **dtype** (data type of its elements).

$$T \in \mathbb{R}^{d_1 \times d_2 \times \dots \times d_n}$$
$$\text{rank}(T) = n \quad (\text{number of axes})$$

Ranks of Tensors

- **Rank-0 (Scalar):** A single number, e.g. 5. No axes.
- **Rank-1 (Vector):** A 1-D array of numbers, e.g. [3, 7, 2, 9]. One axis.
- **Rank-2 (Matrix):** A 2-D grid of numbers (rows & columns). Two axes.
- **Rank-3+ (n-D Tensor):** A stack of matrices, e.g. a color image (height × width × channels), or a batch of images (batch × height × width × channels).

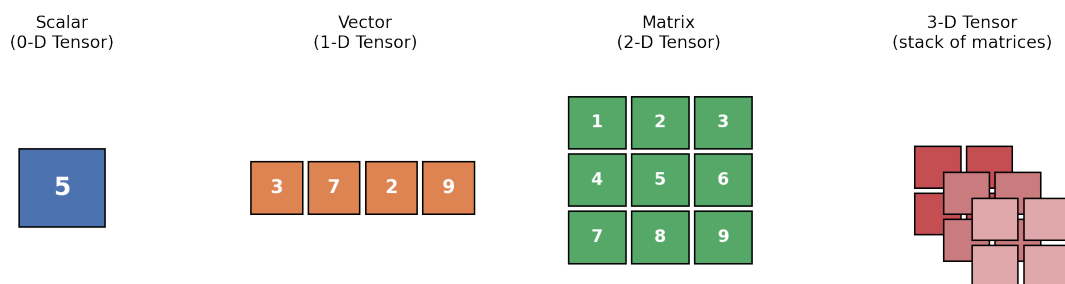


Figure: Visual comparison of tensor ranks — scalar, vector, matrix, and 3-D tensor.

In-depth Examples using NumPy

NumPy's `ndarray` is the standard way to create and manipulate tensors in Python. Below are examples for each rank along with their shape and number of dimensions:

```
import numpy as np

# Rank-0 Tensor (Scalar)
scalar = np.array(5)
print(scalar.ndim, scalar.shape)      # 0 ()

# Rank-1 Tensor (Vector)
vector = np.array([3, 7, 2, 9])
print(vector.ndim, vector.shape)      # 1 (4,)

# Rank-2 Tensor (Matrix)
matrix = np.array([[1, 2, 3],
                  [4, 5, 6],
                  [7, 8, 9]])
print(matrix.ndim, matrix.shape)      # 2 (3, 3)

# Rank-3 Tensor (e.g. stack of matrices / RGB image)
```

```

tensor3d = np.random.randint(0, 255, size=(3, 4, 4))
print(tensor3d.ndim, tensor3d.shape) # 3 (3, 4, 4)

# Rank-4 Tensor (e.g. batch of RGB images)
tensor4d = np.random.rand(32, 64, 64, 3) # 32 images, 64x64, 3 channels
print(tensor4d.ndim, tensor4d.shape) # 4 (32, 64, 64, 3)

# Basic tensor operations
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])
print(a + b) # element-wise addition
print(a @ b) # matrix multiplication
print(a.reshape(4, 1)) # reshaping a tensor

```

In deep learning, data is almost always represented as tensors: an image batch is a rank-4 tensor (batch, height, width, channels), and text/sequence data is often represented as a rank-3 tensor (batch, sequence_length, embedding_dim). Frameworks like TensorFlow and PyTorch use tensors as their core object because GPU hardware is optimized for fast, parallel operations on these multi-dimensional arrays.